



DURBAN UNIVERSITY OF TECHNOLOGY
INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE

FACULTY OF ACCOUNTING AND INFORMATICS
DEPARTMENT OF INFORMATION TECHNOLOGY

2023
MID YEAR SUPPLEMENTARY EXAMINATION

INSTRUCTIONAL PROGRAMME : B INFORMATION & COMMUNICATION TECHNOLOGY
INSTRUCTIONAL OFFERING : INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III
PAPER NUMBER : 1
SUBJECT CODE/S : IPRT301
DATE : 2023/07/21
DURATION : 3 HOURS
TIME : 14H00 - 17H00
TOTAL MARKS : 100
NUMBER OF PAGES : 7 (INCLUDING COVER PAGE)
EXAMINER/S : BOAMAH-ABU, CHARLES (PhD)
MODERATOR/S : IBIDUN OBAGBUWA, CHRISTIANA (PhD)

INSTRUCTIONS/REQUIREMENTS:-

1. This examination is closed book
2. Answer all questions
3. Where stated "briefly", please answer in a few lines
4. Ensure your handwriting is readable

Do not turn the page until permission is given

Question 1

10 marks

In each of the following, indicate whether the statement is **true** or **false**.

1. Integrative programming reduces interoperability of heterogeneous hardware and software resources.
2. An abstract class can extend an interface.
3. The main purpose of an abstract class is code reuse.
4. Any class that extends an abstract class must also be declared abstract.
5. You can create objects of an interface.
6. Method override is resolved at run time.
7. Design patterns are guidelines, not solutions to implement.
8. Organizations can develop their own design patterns.
9. Client-side scripts are safer to execute, compared to server-side.
10. RMI allows different programming languages for the server and client applications.

Question 2

4 marks

Match the following scenarios in software design to the appropriate design patterns. **Write only the number and the corresponding letter.**

	Scenario		Design pattern
1	Separating business objects from operations	A	Singleton
2	Creating instances of subclasses	B	Adaptive
3	Allowing different classes to work together	C	DAO
4	Creating only one instance of a class	D	MVC
		E	Factory

Question 3

11 marks

1. Briefly explain how Integrative programming can reduce software development cost. **(3 marks)**
2. "Choose an interface or abstract class over a concrete class". Explain why. **(2 marks)**
3. What is the main disadvantage of XML, compared to JSON? **(2 marks)**
4. Briefly explain how distributed applications improve scalability. **(2 marks)**
5. "Multithreading is an important feature of distributed applications". Explain why. **(2 marks)**

Question 4

17 marks

The code below is part of an XML document (**debtors.xml**). Each **Client** node contains **id** (a unique identifier), **name** (of client) and **amount** (amount owed). **name** consists of **initial** and **surname**.

```
<?xml version="1.0" encoding="UTF-8"?>
<Account>
  <Client id="1">
    <name>
      <initial>Yaw</initial>
      <surname>Manu</surname>
    </name>
    <amount>2000</amount>
  </Client>
  <Client id="2">
    <name>
      <initial>Zola</initial>
      <surname>Mani</surname>
    </name>
    <amount>1000</amount>
  </Client>
</Account>
```

1. Complete the DTD file (**debtors.dtd**) below. (7 marks)

```
<?xml version='1.0' encoding='UTF-8'?>
<!ELEMENT Account (Client)*>
```

2. Assume there is a 10% interest on amount owed. Complete the XSL document (**debtors.xsl**) below to displays the details of each client as shown on the web page. (8 marks)

http://localhost:8080/Test2Web1/debtors.xml

Debtors' account

Client Id	Initial	Surname	Amount	Interest
1	Y	Manu	2000	200
2	Z	Mani	1000	100

```

....
<xsl:template match="/" ">
    .....
    <xsl:for-each select="_____" ">
    <tr>
        <td>_____</td>
        <xsl:for-each select="_____" ">
        <td>_____</td>
        <td>_____</td>
        </xsl:for-each>
        <td>_____</td>
        <td>_____</td>
    </tr>
    </xsl:for-each>
....

```

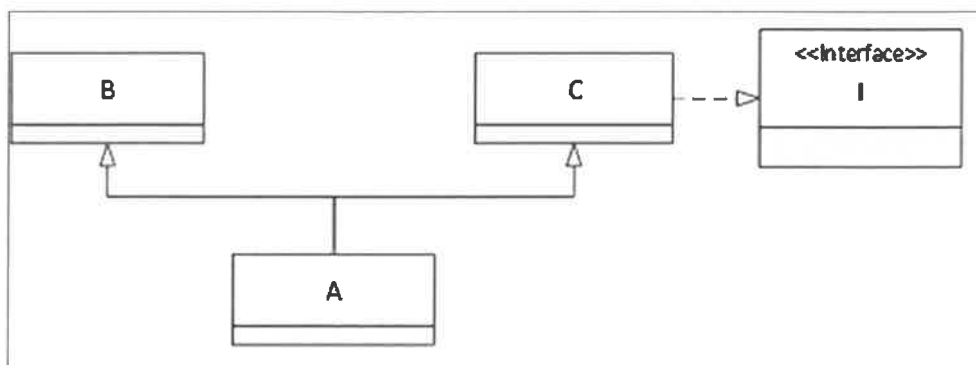
3. Complete the tag (below) to link **debtors.xml** to the structure in **debtors.xsl**. (2 marks)

<?xml-stylesheet type="_____" href="_____"?>

Question 5

6 marks

The UML diagram below shows class **C** implements interface **I** and class **A** inherits classes **B** and **C**. Class **B** contains variables and method implementation while class **C** contains only constants and abstract methods.

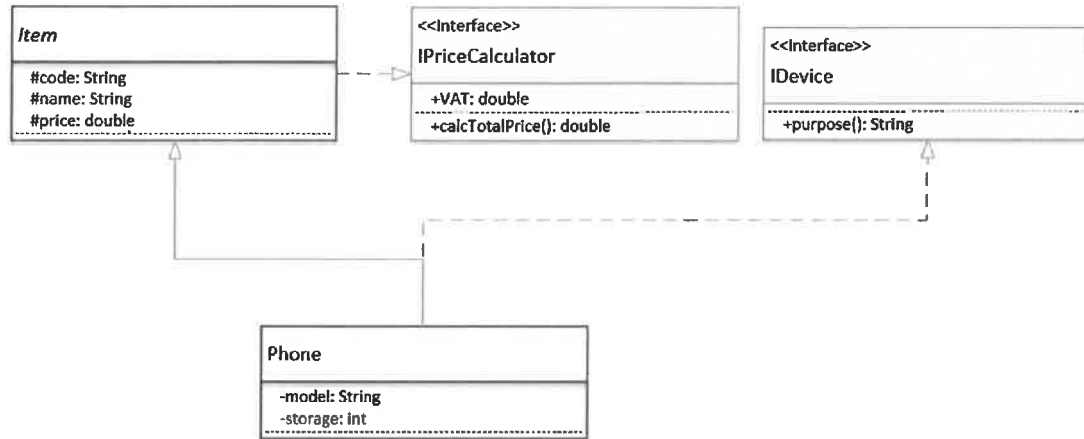


1. Explain how the multiple inheritance can be resolved. (3 marks)
2. What must be the **new** relationship between **C** and **I**? Explain your answer. (3 marks)

Question 6

15 marks

The UML diagram below shows **Item** contains **code**, **name** and **price** of an item. **Phone** contains the **model** and **storage** capacity. **IPriceCalculator** contains **VAT** (a constant, 15%) and **calcTotalPrice()**, which returns the total price as: the price and VAT payable (calculated as 15% of the price of the item). **IDevice** contains **purpose()**, which returns a string.

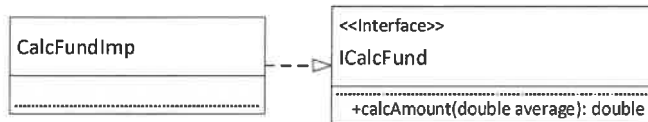


1. Write the method **calcTotalPrice()** in **IPriceCalculator**: (2 marks)
2. Write the implementation of **calcTotalPrice()** in **Item**. (2 marks)
3. Write the class header for **Phone**. (2 marks)
4. Using the classes **Item** and **Phone**, briefly explain the term "polymorphism". (2 marks)
5. In each of the following, state whether the statement is **valid** or **invalid**. Briefly explain your answer.
 - a. `Item i=new Phone();` (2 marks)
 - b. `IPriceCalculator ip=new Phone();` (3 marks)
 - c. `IPriceCalculator ip2=new Item();` (2 marks)

Question 7

17 marks

You are required to create an RMI application. The figure below shows **ICalcFund** (the remote interface). The class **CalcFundImp** implements **ICalcFund**. The method **calcAmount(double average)** takes the average mark of a student and returns the amount to allocate.



1. Write the header for **ICalcFund**. (2 marks)
2. Write the method **calcAmount(double average)** in **ICalcFund**. (3 marks)
3. Write the class header for **CalcFundImp**. (2 marks)
4. Complete the class **Server** (below) which registers and binds an instance of the remote object. Name the remote object **fund**; use the port **1990**. (5 marks)

```

public class Server {
    public static void main(String[] args) _____ {
        CalcFundImp cf=new CalcFundImp();//Create instance
        Registry reg=_____
        _____
    }
}
  
```

5. Complete the class **Client** (below) which calls **calcAmount(double average)** of the remote object with the value 80.0 and displays the result on the screen. Use the local host. (5 marks)

```

public class Client {
    public static void main(String[] args)_____ {
        _____ ic=_____
        System.out.println("The amount is "+ _____)
    }
}
  
```

Question 8

10 marks

Complete the code below which connects to a URL resource (a file "**fees.dtd**" in the folder **webapp** on a **local host**) on port **8080** to **read** the content of the resource and display on the screen. Use **URLConnection**. The program should close all streams.

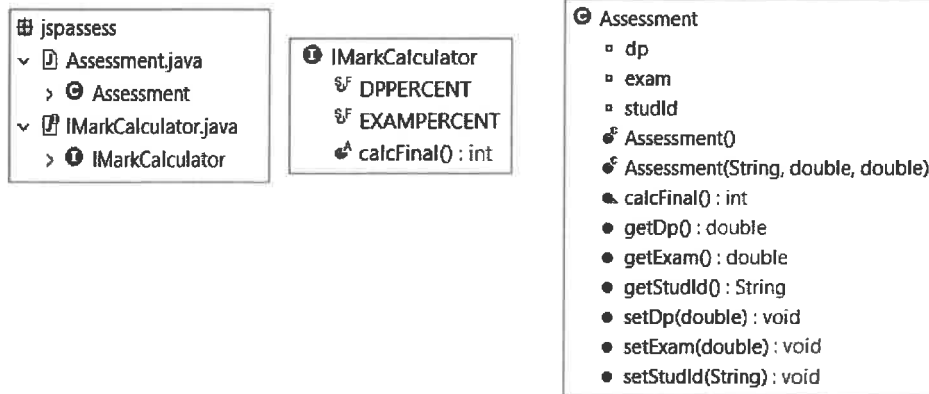
```

public void read() throws Exception{
    _____
    _____
    _____
    _____
    _____
}
  
```

Question 9

10 marks

The package **jspassess** contains the class **Assessment** and interface **IMarkCalculator** (as shown below). **Assessment** contains **dp**, **name** and **studId** (i.e., student Id) and implements **IMarkCalculator**. The method **calcFinal()** returns the final mark (based on **dp** and **exam**).



The package **jspassess** is imported into JSP code to display (as shown below) the **studId**, **dp**, **name** and final mark of a student in an assessment.



Assume the interface and class in **jspassess** have been implemented.

1. Complete the JSP action tag, which creates an instance called **assess** of **Assessment**. (2 marks)

```
<jsp:useBean id="_____" class="_____" />
```

2. Complete the JSP action tags to set the values of the instance **assess**. Assume **studId**="E2012", **dp** =60 and **exam**=60. (3 marks)

```

<jsp:setProperty _____ />
<jsp:setProperty _____ />
<jsp:setProperty _____ />
  
```

3. Write the JSP action tags to get and display the values of the instance **assess**. (3 marks)

```

Student Id=<jsp:getProperty _____ /><br/>
Dp=<jsp: getProperty _____ /><br/>
Exam =<jsp: getProperty _____ /><br/>
  
```

4. Write an EL to call the method **calcDp()** and display the final mark. (2 marks)

```
Final mark=_____
```